

Competency Assessment → Full Platform Manual — Phase 1 → 6.15

A developer · admin · end-user guide to the whole MetryxOne build journey

Track: the competency-to-commercial build journey (Phase 1 = competency framework → Phase 6 = commercial/platform) · **Date:** 22 June 2026
Source of truth: the live code — `backend/routes/*.ts` (route markers) + `backend/services/*.ts` (module functions + file-header phase markers) + the `FF_*` flags actually set in the `Backend API` workflow command. Where a per-subsystem doc holds the exhaustive table list it is named inline (e.g. `docs/EMPLOYABILITY_INDEX.md`).

Phase 1 scope note: the implemented Phase 1 *code* series is **Phase 1 (foundation) + 1.1, 1.2, 1.4, 1.5, 1.6, 1.7** — **there is no 1.3 module, route, or table in code**, and no 1.8–1.15. However, **Phase 1.3 (Competency Data Quality Review)** now exists as a **one-off deliverable**: a read-only audit of the 299-competency genome → `reports/competency_quality_report.md`. It is documented in its own section below, clearly marked as an audit report rather than a running engine.

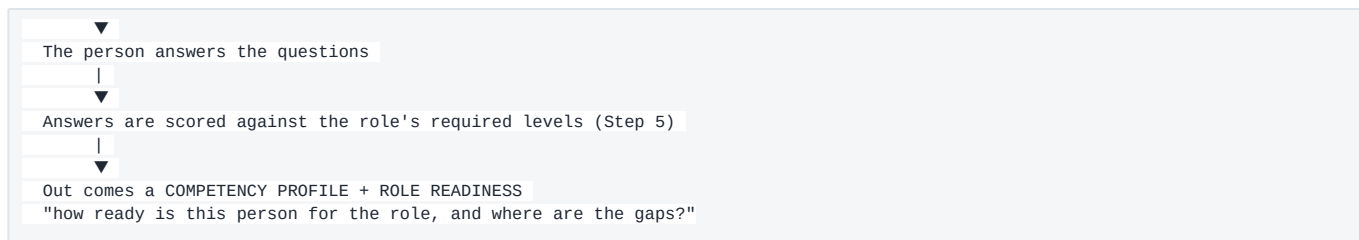
[WARN] Read this about the numbering. This codebase contains **more than one "Phase 1–6" numbering system**. This manual follows the **competency-to-commercial build journey** — the one the competency assessment actually flows into: **Phase 1** competency framework → **Phase 2** competency runtime → **Phase 3** employability/EI → **Phase 4** career intelligence → **Phase 5** employer/enterprise → **Phase 6** commercial/platform/super-admin (up to 6.15). A *separate, older* numbering lives in `docs/phase-history.md` as **"Track A / Track B"** (there, "Phase 2" means Adaptive Benchmarking, "Phase 3" Mobility, etc.). The two are **not the same**; if you cross-reference `phase-history.md`, keep that in mind.

Honesty contract (applies to every phase below). Two axes are always reported **separately**: **Coverage** = does the data exist, and **Confidence/Activation** = is it trustworthy / has it actually been used live. Nothing is fabricated. Phases 1–4 have **exact file-header phase markers** (e.g. `Phase 2.4 -`) and are documented to sub-phase precision. **Phase 5 and Phase 6 sub-numbering is looser** in the code (modules are named, not always numbered) — so those bands are documented by **module name**, and where a `6.x` number is shown it is the number used in the build logs / follow-up tasks, not always a file-header marker.

The end-to-end flow (read this first)

This whole system has **one job**: take a master list of skills and turn it into a real assessment a person can take and be scored on. Here is the entire journey, top to bottom — **each step builds on the one above it**.





The same flow in one sentence

A **master list of skills** (Foundation) is **labelled** (1.1), **governed** (1.2) and **broken into sub-skills** (1.4); each **job role** declares which skills it needs and how much (1.5); those needs become an **assessment blueprint with questions** (1.6); everything is **searchable and manageable** (1.7); and at run-time a person **takes that assessment and gets a readiness score**.

Why it's built in this order (each step needs the one before it)

- You can't **label** skills (1.1) until you have the **skill list** (Foundation).
- You can't say a **role needs a skill** (1.5) until the skills exist and are **active** (1.2).
- You can't build an **assessment** (1.6) until roles have **declared what they need** (1.5).
- You can't **score readiness** until an assessment has been **built and taken**.

Where it stands today (honest status)

- [OK] **Working with real data:** Foundation (299 skills) · 1.1 Type (all 299 labelled) · 1.5 Role Profiles (5 roles) · 1.6 Blueprints · the live run-time (people have taken assessments and been scored).
- [WARN] **Built but empty in the live database:** 1.2 Master and 1.4 Micro — the machinery works, the data just hasn't been loaded yet.
- [WARN] **Thin:** 1.6 question links currently cover only **7 of 299** skills.

Everything below is the **detailed reference** — each module's inner workings, functions, tables, routes, and how to manage it. Use the flow above as your map; drop into a section below only when you need the detail.

How this manual is organised

The whole platform is **one long assembly line**. Each phase takes the output of the phase before it and adds a layer of meaning. Read it top to bottom for the story, or jump to a Part for the detail.

Part	Phase	In one line	Who it's mostly for
Part 1	Phase 1 (+1.1–1.7)	Build the <i>framework</i> : the master skill list, how skills are labelled, which roles need what, and how that becomes an assessment	Admin sets it up; Developer maintains it
Part 2	Phase 2 (2.1–2.4)	The <i>runtime</i> : a real person takes the assessment, answers are scored, a competency profile comes out	End user takes it; Developer runs the engine
Part 3	Phase 3 (3.2–3.12)	<i>Employability / EI</i> : turn the competency profile into an Employability Index , strengths, signals and recommendations	End user reads it; Admin curates rules
Part 4	Phase 4 (4.2–4.12)	<i>Career intelligence</i> : match to roles, find gaps, build a roadmap, track progress	End user (job-seeker)
Part 5	Phase 5 (employer/enterprise)	The <i>employer</i> side: post jobs, find talent, run hiring & workforce intelligence	Employer / enterprise admin
Part 6	Phase 6 (6.1–6.15)	<i>Commercial & platform</i> : payments, plans, quotas, invoices, tenants, partners, and the super-admin command centres	Customer pays; Super-admin runs the business

The journey at a glance (Phase 1 → 6.15)

PHASE 1	Framework	-> a master skill list becomes a role-based ASSESSMENT BLUEPRINT
PHASE 2	Runtime	-> a person TAKES it and is SCORED → COMPETENCY PROFILE + role readiness
PHASE 3	Employability/EI	-> the profile becomes an EMPLOYABILITY INDEX + strengths + signals + recommendations
PHASE 4	Career	-> the person is MATCHED to roles, shown GAPS, given a ROADMAP, tracked over time
PHASE 5	Employer	-> employers POST jobs, FIND talent, run HIRING & WORKFORCE intelligence
PHASE 6	Commercial	-> it all becomes a BUSINESS: plans, payments, quotas, invoices, tenants, command centres

Honest, platform-wide status (one sentence): Phase 1–2 are **proven end-to-end with real scored data** (≈8 competency profiles in the live dev DB); Phases 3–4 are **compose-only engines that re-shape that same runtime data** (so they are as rich as the runtime data underneath them — sparse today); Phase 5 and the **commercial spine** of Phase 6 are **flag-on but un-activated** (no real sales, no completed CAPADEx sessions), while the **newer Phase 6.11–6.15 consoles default OFF** (their flags aren't set in the **Backend API** workflow, so those routes 503 / hide until switched on). Each Part restates its own honest status.

PART 1 — PHASE 1: Building the framework (the back office)

Part 1 is the original, fully-verified detail for the Phase 1 series (Foundation + 1.1–1.7). It is the “set-up” half of the assembly line. Parts 2–6 (the runtime and everything built on it) follow after the Phase 1 summary table.

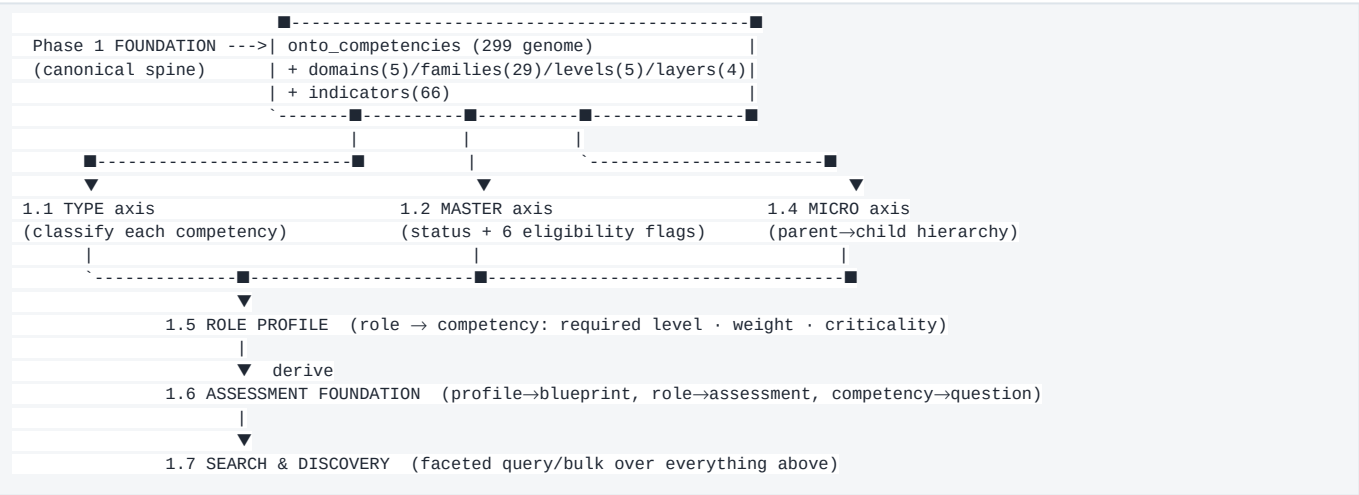
0. The pattern every sub-phase follows

All sub-phases obey the same additive contract:

```
ensure*Schema()    → idempotent lazy DDL (CREATE TABLE/INDEX IF NOT EXISTS); in these services it is called at the top of BOTH read and write
run*Seed()         → idempotent population, ADDITIVE over the genome (never mutates onto_competencies)
get*() read views  → public, requireAuth-gated (flag OFF → 503; flag ON, no session → 401)
admin CRUD + summary → requireAuth + requireSuperAdmin
```

Invariants (apply to all): never mutate the 299-row genome (`onto_competencies`); never fabricate competencies; every id references an EXISTING row; flag-OFF is byte-identical to legacy; reads never throw (degrade to honest empty/zeroed).

The spine → axes shape:



1.7 reads across all layers. The live **assessment runtime** (instances/responses/scores/profiles) consumes **1.6**'s blueprints downstream.

Phase 1 — Foundation (Competency Framework Intelligence spine)

Service: `competency-framework-intelligence.ts` · **Version const:** `COMPETENCY_INTELLIGENCE_VERSION='1.0.0'`

Purpose: expose the raw competency genome as a governed, read-only “canonical spine” that every later axis hangs off.

Modules (functions): `CANONICAL_SPINE` (static descriptor) · `getMasterCompetencies` · `getRoleRequirements` · `getCompetencyLevels` · `getIndicators` · `getTaxonomy` · `buildCompetencyCrosswalk` · `getFrameworkReadiness` (admin).

Endpoints (public, requireAuth):

- `GET /api/competency-intelligence/spine` — framework descriptor

- GET /competencies?domain_id&family_id&q&limit — genome list
- GET /role-requirements?role= — role→competency requirements (read)
- GET /levels · GET /indicators · GET /taxonomy · GET /crosswalk

Tables: `onto_competencies` (299), `onto_domains` (5), `onto_families` (29), `onto_proficiency_levels` (5), `onto_layers` (4), `onto_indicators` (66), `onto_role_weights` (35), `onto_aliases` (19).

Detailed workflow (step by step):

1. **Spine descriptor** — GET /spine returns the static `CANONICAL_SPINE` descriptor (namespaces, table inventory, version `1.0.0`); no DB read.
2. **Genome read** — `getMasterCompetencies` reads the `onto_competencies` genome via the ontology service (`listCompetencies`) with `domainId/familyId/search/limit` filters, and returns `canonical_count` (a `safeCount` of `onto_competencies`) for coverage.
3. **Taxonomy assembly** — `getTaxonomy` returns 5 tiers — `industry · function · department · role_family · role` — and each tier carries BOTH its `onto_*` and `ont_*` table reference (e.g. `onto_subfunctions` ↔ `ont_departments`); the cross-namespace mapping lives here, not in the competency read.
4. **Levels & indicators** — `getCompetencyLevels` returns the proficiency levels (with layers/anchors) and `getIndicators` returns the 66 indicators, each as a separate payload (not joined onto every competency row).
5. **Role requirements** — `getRoleRequirements` resolves a role string to an ONET code via `resolveBestOntRole`, then pulls weighted requirements through `role-crosswalk` (honest null/empty when unresolved — never fabricated).
6. **Crosswalk** — `buildCompetencyCrosswalk` emits the cross-namespace mapping for GET /crosswalk.
7. **Readiness roll-up** — `getFrameworkReadiness` iterates `ASSET_SPECS` (30+ declared tables), runs `safeCount` on each, and derives per-asset Coverage (row count) vs Confidence* (qualitative provenance) → the master admin readiness report.

Live state: [OK] populated, end-to-end functional.

Phase 1.1 — Competency Type Classification

Service: `competency-type-classification.ts` · **Version:** `phase-1.1`

Purpose: add a **type axis** — classify all 299 competencies into 5 canonical types.

Modules: `COMPETENCY_TYPES` (5: behavioral · cognitive · functional · technical · future_skills) · `FUTURE_SKILLS_LEXICON` / `TECHNICAL_LEXICON` / `TECHNICAL_FAMILIES` (classifier inputs) · `classifyCompetency(row)` (deterministic rule engine) · `ensureCompetencyTypeSchema` · `runCompetencyTypeSeed` · `getCompetencyTypes` · `getCompetencyTypeMap` · `getClassificationReport`.

Endpoints:

- GET /types — the 5-row type master
- GET /type-map?type&needs_review=1 — every competency's classification
- GET /admin/.../readiness · GET /admin/.../classification-report

Tables: `onto_competency_types` (5) · `onto_competency_type_map` (299).

Detailed workflow (step by step):

1. **Schema** — `ensureCompetencyTypeSchema` (idempotent `CREATE ... IF NOT EXISTS`, called at the top of every read and write function) creates `onto_competency_types` (5-row master) + `onto_competency_type_map` (one row per competency).
2. **Seed types** — `runCompetencyTypeSeed` upserts the 5 canonical types, then iterates every `onto_competencies` row through `classifyCompetency`.
3. **Classification cascade** (`classifyCompetency`, deterministic — first match wins):
 - a. Normalise name + definition (lowercase, strip punctuation).
 - b. `FUTURE_SKILLS_LEXICON` hit on **name** → `future_skills` (Medium confidence); on **definition** only → Low confidence.
 - c. `family_id` ∈ `TECHNICAL_FAMILIES` → `technical` (High); else `TECHNICAL_LEXICON` match → `technical` (Medium/Low).
 - d. Inherit the existing `scientific_type` for behavioral / cognitive / functional (High).
 - e. No rule matches → default `behavioral`, Low confidence, `needs_review = true`.
4. **Persist** — write the assigned type + confidence + `needs_review` to `onto_competency_type_map` (the genome is never touched).
5. **Report** — `getClassificationReport` returns coverage, per-type distribution, confidence mix, `needs_review` count, and honest content-gap findings.

Live state: [OK] 299/299 classified. **Honest gap:** Future Skills = 0, Technical sparse (a *content* gap the classifier surfaces, not an engine fault).

Phase 1.2 — Competency Master Enhancement

Service: `competency-master.ts` · **Version:** `phase-1.2`

Purpose: add an **eligibility/status axis** — the enhanced competency entity (status + which downstream products may consume each competency).

Modules: `ELIGIBILITY_FLAGS` (6) · `COMPETENCY_STATUSES` (active/inactive/deprecated) · `ensureCompetencyMasterSchema` · `runCompetencyMasterSeed` · `getCompetencyMaster` · `getCompetencyMasterById` · `updateCompetencyMaster` · `getCompetencyMasterSummary`.

Endpoints:

- GET /master?q&type&status&limit
- GET /admin/.../master-summary
- PATCH /admin/.../master/:id — edit status + 6 flags (404 if id unknown; never creates)

6 eligibility flags: `assessment_eligible` · `ei_eligible` · `career_builder_eligible` · `employer_eligible` · `learning_eligible` · `future_ready_eligible`.

Tables: `onto_competency_master_ext` (extension row per competency; `source=curated|default`).

Detailed workflow (step by step):

- 1. **Schema** — `ensureCompetencyMasterSchema` creates `onto_competency_master_ext` (one extension row per competency).
- 2. **Seed defaults** — `runCompetencyMasterSeed` inserts one ext row per genome competency: maps the genome's existing `deprecated` flag → `status` (`deprecated` or `active`), sets all 6 eligibility flags `true`, stamps `source='default'`.
- 3. **Read view** — `getCompetencyMaster` composes `onto_competencies` + `onto_competency_type_map` + `onto_competency_master_ext` into one `CompetencyMasterRow`; filters by `q/type/status`.
- 4. **Curate** — `updateCompetencyMaster` (`PATCH /master/:id`) updates status + the 6 flags for one id and flips `source='curated'`; returns 404 if the id is unknown — it never creates a competency.
- 5. **Report** — `getCompetencyMasterSummary` returns coverage, status breakdown, per-flag eligibility counts, and curated-vs-default provenance.

Live state: [WARN] `onto_competency_master_ext = 0` in the live shared DB → engine + routes live, but no data (seed not present here). This is the #1 activation gap.

Phase 1.3 — Competency Data Quality Review (audit deliverable, not a code module)

Deliverable: `reports/competency_quality_report.md` · **Generated:** 22 June 2026 · **Type:** one-off, read-only data-quality audit (no engine, no route, no table).

Why it's here: the Competency Framework Intelligence code never shipped a Phase 1.3 (the markers in `competency-intelligence.ts` jump 1.2 → 1.4). This report is the **missing 1.3 deliverable** — a manual audit of the **299-row `onto_competencies` genome** (the canonical master that Phase 1 builds and Phase 2 consumes). It is **read-only**: nothing in the database was merged, renamed, or deleted, and per project policy every cleanup recommendation **stops for approval** before any mutation.

Plain-English overview: "Is the master list of ~300 competencies clean?" The answer: **no literal duplicates, no deprecated rows, no (duplicate) markers** — but there is **naming inconsistency** (word-order / qualifier variants of the same idea, e.g. *Building Trust* vs *Trust Building*) and **classification inconsistency** (the same idea filed under different `scientific_types`, e.g. *Delegation* behavioral vs *Task Delegation* functional).

What it found (scorecard):

Check	Result
Competencies reviewed	299
Deprecated flags / exact-duplicate names / (duplicate) markers	0 / 0 / 0
Name-variant duplicate groups	5 confirmed + 7 lower-confidence
Near-duplicate pairs flagged by similarity	73 (most are genuinely <i>distinct</i> → RETAIN)
Genuine classification conflicts	3
Merge groups recommended	6 high-confidence + 7 medium-confidence

Audience views:

- **End user / leadership:** the competency catalogue is structurally sound; the work left is editorial (consolidate ~6–13 near-synonym labels) and governance (pick one type per construct). No data was changed.
- **Admin / framework curator:** §3 lists exact merge recommendations (retain-which + why); §4 lists the 3 type/domain conflicts to reconcile *before* merging; §5 lists the 73 similar-but-distinct pairs deliberately kept apart. Apply via the **Phase 1.2 PATCH `/master/:id`** (status/deprecate) and **Phase 1.4 hierarchy** (parent-child) surfaces — there is no bulk "merge" endpoint, so changes are curated, reversible, and approval-gated.
- **Developer:** method = name normalization → exact-name + token-set + fuzzy similarity (string ratio + token Jaccard), then **manual adjudication** against `domain_id` / `family_id` / `scientific_type`. **Critical limitation:** every `definition` is an auto-generated placeholder ("X — *canonical competency in the Y family.*") carrying **no real semantic content**, so true meaning-based de-dup is impossible from data alone — **authoring real definitions is the single highest-value cleanup action** (reported as a Coverage gap: the field exists, but Confidence in data for dedup is zero).

Honest status: [OK] **complete as an audit** (299 reviewed, **0 modified**). [AMBER] **No remediation applied yet** — all merges/reconciliations remain recommendations pending approval. Scope was limited to `onto_competencies`; sibling tables (`competency_library`, `competency_catalog`, `ont_competencies`, ...) are empty in this environment and were excluded.

Phase 1.4 — Micro Competency Framework

Service: `micro-competency.ts` · **Version:** `phase-1.4`

Purpose: add a **hierarchy axis** — parent→child (micro) structure over the genome. A child is EITHER a real linked competency OR a named-only micro (honestly flagged).

Modules: `slugify` · `ensureMicroCompetencySchema` · `getMicroFramework` (nested) · `getMicroMapping` (flat) · `createMicroRelationship` / `update` / `delete` · `SEED_FRAMEWORK` · `runMicroCompetencySeed` · `getMicroFrameworkSummary`.

Endpoints:

- `GET /micro-framework?parent_id&q&active` · `GET /micro-mapping`

- GET /admin/.../micro-framework/summary
- POST (create — validates parent & linked child EXIST) · PATCH /:id (toggle/reorder/relabel) · DELETE /:id

Tables: `onto_competency_hierarchy` (parent_competency_id → child_competency_id | micro_label).

Detailed workflow (step by step):

1. **Schema** — `ensureMicroCompetencySchema` creates `onto_competency_hierarchy` with a `chk_hier_no_self` check (parent ≠ child).
2. **Seed** — `runMicroCompetencySeed` populates the curated `SEED_FRAMEWORK` groups (Communication, Leadership, Problem-Solving).
3. **Create** (`createMicroRelationship`):
 - **Linked child** — `child_competency_id` FK must reference a real `onto_competencies` row; the label is locked to the canonical name (`linked: true`).
 - **Named-only child** — no FK; stores `micro_label` + `micro_slug` (via `slugify`), flagged `linked: false` (honest).
 - Parent existence is validated; self-reference is rejected by the check constraint.
4. **Read** — `getMicroFramework` (nested parent→children) and `getMicroMapping` (flat) with `parent_id/q/active` filters.
5. **Mutate** — `PATCH /:id` toggles active / reorders / relabels; `DELETE /:id` removes a relationship (genome untouched, fully reversible).
6. **Report** — `getMicroFrameworkSummary` returns coverage + linked-vs-named provenance.

Live state: [WARN] `onto_competency_hierarchy = 0` in the live DB (was 12 in the 19-Jun audit env). Engine functional; relationships need authoring/seeding.

Phase 1.5 — Role Competency Profile Engine

Service: `role-competency-profile.ts` · **Version:** `phase-1.5`

Purpose: add the **role→competency requirement layer** (required level · weight · criticality). Powers 3 deliverables: Role Profile, Role Matrix, Role Readiness.

Modules: `CRITICALITY_TIERS` (critical/important/desirable/optional) · `READINESS_BANDS` · `readinessBand(score)` · `roleFit(score, blockingGaps)` · `getRoleProfiles` · `getRoleProfile` · `getRoleCompetencyMatrix` · `getRoleReadiness` · `CRUD` · `runRoleCompetencyProfileSeed` · `getRoleCompetencyProfileSummary`.

Endpoints:

- GET /role-profiles?role_id&q&active — nested role→competency requirements
- GET /role-matrix — roles × competencies grid
- GET /role-readiness/:roleId?actuals=comp:level,... — weighted gap of actual vs required (no actuals → required structure only, readiness honestly unmeasured)
- GET /admin/.../role-profiles/summary · /role-profiles/role/:roleId · POST / PATCH /:id / DELETE /:id

Tables: `onto_role_competency_profiles` (14), `onto_role_weights` (35); links to `onto_roles` (5).

Detailed workflow (step by step):

1. **Seed** — `runRoleCompetencyProfileSeed` populates curated per-role requirements in `onto_role_competency_profiles` (validates that role AND competency exist).
2. **Create/edit** — admin `POST/PATCH /:id` set `required_level` (1–5), `weight` (0–100), `criticality` ∈ {critical, important, desirable, optional}, rationale, active; both references are validated.
3. **Profile & matrix** — `getRoleProfiles` returns nested role→competency requirements; `getRoleCompetencyMatrix` returns the roles × competencies grid.
4. **Readiness computation** (`getRoleReadiness/:roleId?actuals=comp:level,...`):
 - per competency `attainment = min(actual_level / required_level, 1)`
 - `readiness_score = Σ(attainment × weight) / Σ(assessed_weight) × 100`
 - **blocking gaps** = count of `criticality='critical'` competencies where `actual_level < required_level`
 - band the score via `readinessBand`; `roleFit`: ≥85 strong · ≥70 good · ≥50 partial · <50 low — **capped at partial whenever blockingGaps > 0**, regardless of score
 - no `actuals` supplied → returns the required structure only, readiness honestly unmeasured.
5. **Report** — `getRoleCompetencyProfileSummary`: coverage, weight integrity, criticality mix, findings.

Live state: [OK] 14 profiles / 35 weights / 3 role→assessment maps. **Gap:** breadth (only 5 roles).

Phase 1.6 — Assessment Foundation Mapping

Service: `assessment-foundation-mapping.ts` · **Version:** `phase-1.6`

Purpose: connect the genome to the assessment surface **without redesigning any assessment workflow**. Three deliverables:

1. **Competency → Question** (`onto_competency_question_map`)
2. **Role → Assessment** (`onto_role_assessment_map` → `onto_assessment_blueprints`)
3. **Competency Profile → Blueprint** (`onto_assessment_blueprints` + `onto_blueprint_competency_map`)

Modules: `getBlueprints` / `getBlueprint` / blueprint CRUD · `addBlueprintCompetency` / `deleteBlueprintCompetency` · `getRoleAssessmentMap` / `createRoleAssessment` / `deleteRoleAssessment` · `getCompetencyQuestionMap` / `createCompetencyQuestion` / `getMappingGrid` / `bulkMapCompetencyQuestions` / `deleteCompetencyQuestion` · **`deriveBlueprintsFromProfiles`** · **`deriveCompetencyQuestionMap`** (auto-derivation) · `runAssessmentFoundationSeed` · `getAssessmentFoundationSummary`.

Endpoints: GET /blueprints, /blueprints/:id, /role-assessments, /competency-questions; admin `assessment-foundation/summary` + full CRUD for blueprints, blueprint-competencies, role-assessments, competency-questions.

Tables: `onto_assessment_blueprints` (6), `onto_blueprint_competency_map` (28), `onto_role_assessment_map` (3), `onto_competency_question_map` (25 rows).

Workflow (key — this is where 1.5 feeds assessment):

```
1.5 role profiles --deriveBlueprintsFromProfiles--> onto_assessment_blueprints
                                     + onto_blueprint_competency_map
role --createRoleAssessment--> onto_role_assessment_map --> blueprint (is_primary)
competency --deriveCompetencyQuestionMap / bulkMap--> onto_competency_question_map --> existing question bank
```

Detailed workflow (step by step):

- 1. **Derive blueprints** — `deriveBlueprintsFromProfiles` reads each Phase 1.5 role profile and projects it verbatim into `onto_assessment_blueprints` + `onto_blueprint_competency_map` (each blueprint competency carries the profile's weight).
- 2. **Map roles → assessments** — `createRoleAssessment` writes `onto_role_assessment_map`, marking one blueprint per role `is_primary`.
- 3. **Map competencies → questions** — `deriveCompetencyQuestionMap` (or `bulkMapCompetencyQuestions`) links genome competency ids to rows in `competency_question_templates` via `onto_competency_question_map`. If no questions exist, the map stays empty — never seeded with placeholders.
- 4. **Read views** — `getBlueprints/getBlueprint` return a `BlueprintView` carrying `weight_total` and `weight_balanced` (a boolean check that the weights sum ≈ 100 ±0.5).
- 5. **Integrity reporting** — `getAssessmentFoundationSummary` counts blueprints, blueprint-competencies, unbalanced blueprints (reported as-is, **never auto-normalised**), role coverage, and competency-question links across N distinct competencies vs questions available.
- 6. **Manual CRUD** — blueprints, blueprint-competencies, role-assessments and competency-questions each have admin `POST/PATCH/DELETE`, all validating that referenced rows exist.

Live state: [OK] blueprints + maps populated. **Honest gap:** `onto_competency_question_map` has 25 rows but covers only **7 distinct competencies of 299** — the bulk of competencies are not yet directly measurable.

Phase 1.7 — Search & Discovery

Service: `competency-search.ts` · (no version const — cosmetic only)

Purpose: faceted query + bulk operations across the entire framework (genome + type + master + micro + role).

Modules: `searchCompetencies(opts)` (faceted, filter groups) · `getCompetenciesByIds` · `getSearchFacets` · `searchMicroCompetencies` · `getSearchSummary` · `bulkOperation`.

Tables: none of its own — reads across the layers above.

Detailed workflow (step by step):

- 1. **Facets** — `getSearchFacets` builds facet groups — types, domains, families, taxonomy (industries/functions/departments/roles) and attribute lists — from live data, each with counts (there is no `status` facet).
- 2. **Search** — `searchCompetencies(opts)` applies the selected filter groups + pagination across the composed layers. Taxonomy filters (industry/function/department) resolve by joining `onto_role_competency_profiles` → `onto_roles` → `onto_role_families` → `onto_subfunctions` → `onto_functions` → `onto_industries`; micro-counts and role-relevance come from correlated subqueries.
- 3. **Targeted lookups** — `getCompetenciesByIds` (resolve a set of ids) and `searchMicroCompetencies` (micro layer).
- 4. **Summary** — `getSearchSummary`: totals (active/deprecated), typed vs untyped, domains/families, micro count, taxonomy counts, and a type breakdown.
- 5. **Bulk operation** — `bulkOperation` (admin `POST /search/bulk`) supports exactly two operations: `export` (returns the selected competencies) and `assign_type` (sets the Phase 1.1 type for a set of ids, stamping `provenance='manual_bulk'`). It never changes master status and never imports new competencies.

Live state: [OK] functional over the live 299 genome.

Reporting Layer (admin analytics — every sub-phase has one)

Reporting is **not a separate phase** — each sub-phase ships its own read-only report function, and Phase 1's `getFrameworkReadiness` is the **master roll-up** across all of them. Every report follows the same honesty contract: **Coverage (does data exist) and Confidence (is it trustworthy/sufficient) are reported as SEPARATE axes**, every report returns plain-language `findings[]`, empty tables produce an explicit "run the seed ..." instruction (never a fabricated number or placeholder).

Master report — Framework Readiness (roll-up)

Module: `getFrameworkReadiness` (Phase 1 foundation) · **Endpoint:** `GET /api/admin/competency-intelligence/readiness`
Output: { `generated_at`, `spine`, `assets[]`, `totals` } where each `asset` = { `spec`, `coverage_rows`, `status` } and `status` ∈ `consumable` | `empty_pending_import` | `unknown` (derived from live row count). `totals` = { `total_assets`, `consumable`, `empty_pending_import`, `unknown` }.
Workflow: `safeCount` each declared framework table → derive a per-asset status → aggregate. This is the one call that tells an admin *which sub-phases are data-active vs empty* (e.g. it flags 1.2 / 1.4 as `empty_pending_import` today).

Per-sub-phase reports

Phase	Module	Endpoint	Reports
-------	--------	----------	---------

1.1	<code>getClassificationReport</code>	<code>/admin/.../classification-report</code>	coverage · type distribution · classifier confidence · <code>needs_review</code> count · honest findings
1.2	<code>getCompetencyMasterSummary</code>	<code>/admin/.../master-summary</code>	coverage · status breakdown · per-module eligibility counts (6 flags) · curated-vs-default provenance · findings
1.4	<code>getMicroFrameworkSummary</code>	<code>/admin/.../micro-framework/summary</code>	coverage · linked-vs-named-only provenance · findings
1.5	<code>getRoleCompetencyProfileSummary</code>	<code>/admin/.../role-profiles/summary</code>	coverage · weight integrity · criticality mix · findings
1.6	<code>getAssessmentFoundationSummary</code>	<code>/admin/.../assessment-foundation/</code>	<code>blueprints_total</code> , <code>blueprint_competencies_total</code> , <code>unbalanced_blueprints</code> + <code>blueprint_integrity[]</code> (weights sum to 100 ±0.5, reported as-is, never auto-normalised), <code>roles_total/roles_mapped/role_coverage_pct</code> , competency-question links across N distinct competencies vs <code>questions_available</code> , source provenance, findings
1.7	<code>getSearchSummary</code>	search summary	<code>competencies_total/active/deprecated</code> , <code>typed</code> vs <code>untyped</code> , domains/families, <code>micro_competencies</code> , taxonomy (industries/functions/departments/roles), <code>type_breakdown[]</code> , findings

How the reports behave honestly (examples pulled from the code)

- **1.6 empty blueprints** → finding: "No assessment blueprints yet — run the seed to derive blueprints from the Phase 1.5 role competency profiles."
- **1.6 empty question bank** → finding: "Competency question mapping is EMPTY because no questions exist in `competency_question_templates` yet — the mapping is infrastructure only; it is never seeded with placeholder questions."
- **1.6 unbalanced weights** → counted and listed, "reported as-is, never auto-normalised."
- **1.7 untyped competencies** → surfaced under an "untyped" filter rather than force-classified.
- **1.4 no micros** → "No micro-competencies defined yet — micro search returns empty (honest, never seeded with placeholders)."

Reporting data flow

```

each sub-phase tables --get*Summary()--> per-phase admin report (coverage/confidence/findings)
|
onto_* table row counts --safeCount--> getFrameworkReadiness --> master readiness roll-up
|
▼
SuperAdmin UI (CFI tab / RoleCompetencyProfilePanel) renders the reports

```

Live reporting state: all report endpoints are live (401-gated). The readiness roll-up currently reports **1.2 (onto_competency_master_ext)** and **1.4 (onto_competency_hierarchy)** as **empty/pending**, and 1.6 as active-but-thin (7 distinct competencies mapped) — i.e. the reporting layer itself already tells the honest activation story.

Module Reference — Functionality · How to manage · How to access

Access prerequisites (apply to every module):

- **Flag:** `FF_COMPETENCY_FRAMEWORK_INTELLIGENCE=1` must be set (it already is, in the **Backend API** workflow). Flag OFF → all endpoints 503 and the UI panels hide.
- **Public reads** (`/api/competency-intelligence/*`): require a logged-in session (`requireAuth`). No session → 401.
- **Admin endpoints** (`/api/admin/competency-intelligence/*`): require a **super-admin** session (`requireAuth` + `requireSuperAdmin`). Log in as `support@metryxone.com` (2FA-gated; in dev read the code from the `mfa_codes` table).
- **UI home:** SuperAdmin Dashboard → the Competency Framework Intelligence area. Each module has its own panel under `frontend/src/components/superadmin/`.
- **Manage via seed:** run from the repo as `cd backend && npx tsx scripts/<seed>.ts`. Seeds are **idempotent and additive** (safe to re-run; never mutate or duplicate the genome). Restart **Backend API** after a new route is added (not needed after a seed).

Phase 1 — Foundation (spine)

- **Functionality:** governs and exposes the 299-competency genome + taxonomy (domain→family→competency), proficiency levels, indicators, aliases and the crosswalk as a single read-only "canonical spine" every other module hangs off.
- **How to manage:** the genome is curated via SQL migration (the `onto.*` library; there is no runtime genome seed and no genome edit endpoint — it is read-only by design). Content changes go through a migration. The admin **readiness report** is how you monitor it.
- **How to access:** `GET /api/competency-intelligence/{spine,competencies,role-requirements,levels,indicators,taxonomy,crosswalk}` · admin `GET /api/admin/competency-intelligence/readiness`. **UI:** `CompetencyFrameworkIntelligencePanel.tsx`.

Phase 1.1 — Competency Type Classification

- **Functionality:** assigns each of the 299 competencies one of 5 types (behavioral · cognitive · functional · technical · future_skills) using deterministic family/lexicon rules, flagging low-confidence rows for review.
- **How to manage:** `npx tsx scripts/seed-competency-types.ts` (re-run to re-classify after rule/genome changes). There is no per-row edit endpoint — classification is seed-driven; review quality through the classification report (`needs_review`).
- **How to access:** `GET /types`, `GET /type-map?type=&needs_review=1` · admin `GET /api/admin/competency-intelligence/classification-report`. **UI:** surfaced inside the CFI panel.

Phase 1.2 — Competency Master Enhancement

- **Functionality:** adds a status (active/inactive/deprecated) and 6 product-eligibility flags (assessment / EI / career-builder / employer / learning / future-ready) to each competency, controlling which downstream products may consume it.
- **How to manage:** seed defaults with `npx tsx scripts/seed-competency-master.ts`, then curate each competency via `PATCH /api/admin/competency-intelligence/master/:id` (stamps `source=curated`; 404 if the id is unknown — never creates a competency). Easiest in the panel.
- **How to access:** `GET /master?q&type&status&limit` · admin `GET .../master-summary`, `PATCH .../master/:id`. **UI:** `CompetencyMasterPanel.tsx`.

Phase 1.4 — Micro Competency Framework

- **Functionality:** builds a parent→child (micro) hierarchy over the genome; a child is either a linked existing competency or a named-only micro (honestly flagged).
- **How to manage:** seed with `npx tsx scripts/seed-micro-competency.ts`; then add/edit relationships via admin `POST` (create — validates parent & linked child exist), `PATCH /:id` (toggle active / reorder / relabel), `DELETE /:id`. All reversible; genome untouched.
- **How to access:** `GET /micro-framework?parent_id&q&active`, `GET /micro-mapping` · admin `GET .../micro-framework/summary` + `POST/PATCH/DELETE`. **UI:** `CompetencyMicroFrameworkPanel.tsx`.

Phase 1.5 — Role Competency Profile Engine

- **Functionality:** defines per-role competency requirements (required level · weight · criticality) and computes Role Profile, Role Competency Matrix, and weighted Role Readiness (actual vs required).
- **How to manage:** seed with `npx tsx scripts/seed-role-competency-profile.ts`; then manage requirements via admin `POST` (validates role AND competency exist), `PATCH /:id` (level/weight/criticality/rationale/active), `DELETE /:id`.
- **How to access:** `GET /role-profiles?role_id&q&active`, `GET /role-matrix`, `GET /role-readiness/:roleId?actuals=comp:level,...` · admin `GET .../role-profiles/summary`, `.../role-profiles/role/:roleId` + CRUD. **UI:** `RoleCompetencyProfilePanel.tsx`.

Phase 1.6 — Assessment Foundation Mapping

- **Functionality:** connects the genome to the assessment surface via 3 mappings — Profile→Blueprint, Role→Assessment, Competency→Question — without redesigning any assessment flow.
- **How to manage:** seed with `npx tsx scripts/seed-assessment-foundation-mapping.ts` (auto-derives blueprints from Phase 1.5 profiles, and competency→question links once real questions exist in `competency_question_templates`). Then admin-manage blueprints, blueprint-competencies, role-assessments and competency-questions via their CRUD endpoints (and bulk-map). Blueprint weights are validated to sum to 100 but **never auto-normalised**.
- **How to access:** `GET /blueprints`, `/blueprints/:id`, `/role-assessments`, `/competency-questions` · admin `GET .../assessment-foundation/summary` + full CRUD (blueprints, blueprint-competencies, role-assessments, competency-questions). **UI:** `AssessmentFoundationMappingPanel.tsx`.

Phase 1.7 — Search & Discovery

- **Functionality:** faceted search and bulk operations across all the layers above (genome + type + master + micro + role taxonomy), with facet counts and an "untyped" filter.

- **How to manage:** no seed of its own (it reads the other modules); admins run cross-cutting actions via `bulkOperation` over a result set.
- **How to access:** the search/facets functions (`searchCompetencies`, `getSearchFacets`, `searchMicroCompetencies`, `getSearchSummary`) power the search boxes embedded in the CFI panels. **UI:** search inputs within the CFI / panel surfaces.

Access at a glance

Module	UI panel (<code>components/superadmin/</code>)	Public read base (<code>/api/competency-intelligence/</code>)	Admin base (<code>/api/admin/competency-intelligence/</code>)	Seed script (<code>backend/scripts/</code>)
Foundation	<code>CompetencyFrameworkIntelligencePanel</code>	<code>/spine/, /competencies, /taxonomy, /levels, /indicators, /crosswalk</code>	<code>/readiness</code>	(SQL migration — no runtime seed)
1.1 Type	(in CFI panel)	<code>/types, /type-map</code>	<code>/classification-report</code>	<code>seed-competency-types.ts</code>
1.2 Master	<code>CompetencyMasterPanel</code>	<code>/master</code>	<code>/master-summary, PATCH /master/:id</code>	<code>seed-competency-master.ts</code>
1.4 Micro	<code>CompetencyMicroFrameworkIntelligencePanel</code>	<code>/micro-framework, /micro-mapping</code>	<code>/micro-framework/summary + CRUD</code>	<code>seed-micro-competency.ts</code>
1.5 Role Profile	<code>RoleCompetencyProfilePanel</code>	<code>/role-profiles, /role-matrix, /role-readiness/:roleId</code>	<code>/role-profiles/summary + CRUD</code>	<code>seed-role-competency-profile.ts</code>
1.6 Assessment Foundation	<code>AssessmentFoundationMappingPanel</code>	<code>/blueprints, /role-assessments, /competency-questions</code>	<code>/assessment-foundation/summary + CRUD</code>	<code>seed-assessment-foundation-mapping.ts</code>
1.7 Search	<code>CompetencySearchPanel</code>	<code>/search, /search/summary, /search/facets, /search/micro-competencies</code>	<code>/search/bulk</code>	(none — reads other modules)

Module file paths (frontend · super-admin · users)

How the layers map: every Phase 1.x module is implemented in **one backend route file** (`backend/routes/competency-intelligence.ts`) plus a **per-module service**, and is operated from a **dedicated super-admin panel**. These modules are **super-admin / back-office curation tools** — they have **no direct end-user page**. The genome they curate is consumed *indirectly* by the user-facing competency experience, which calls the separate `/api/competency/*` runtime endpoints (not the `/api/competency-intelligence/*` endpoints documented here).

Backend paths

Module	Route file	Service file
Foundation	<code>backend/routes/competency-intelligence.ts</code>	<code>backend/services/competency-framework-intelligence.ts</code>
1.1 Type	<code>backend/routes/competency-intelligence.ts</code>	<code>backend/services/competency-type-classification.ts</code>
1.2 Master	<code>backend/routes/competency-intelligence.ts</code>	<code>backend/services/competency-master.ts</code>
1.4 Micro	<code>backend/routes/competency-intelligence.ts</code>	<code>backend/services/micro-competency.ts</code>
1.5 Role Profile	<code>backend/routes/competency-intelligence.ts</code>	<code>backend/services/role-competency-profile.ts</code>
1.6 Assessment Foundation	<code>backend/routes/competency-intelligence.ts</code>	<code>backend/services/assessment-foundation-mapping.ts</code>
1.7 Search	<code>backend/routes/competency-intelligence.ts</code>	<code>backend/services/competency-search.ts</code>

Super-admin frontend paths

All panels live under `frontend/src/components/superadmin/` and are lazy-registered (and flag-gated by `cfiEnabled`) in `frontend/src/components/SuperAdminDashboard.tsx`. The `tab id` is how you deep-link to the panel within the dashboard.

How to navigate there (step by step):

1. Open the app and go to the Super Admin login (SPA screen `super-admin`).

2. Sign in as `support@metryxone.com / admin123` → complete the emailed 2FA code (in dev, read it from the `mfa_codes` table) → you land on SPA screen `admin-dashboard` (`frontend/src/components/SuperAdminDashboard.tsx`).
3. In the dashboard's left navigation open the **Competency** group; click the module whose **tab id** is listed below. (These tabs only appear when `FF_COMPETENCY_FRAMEWORK_INTELLIGENCE=1` — `cfiEnabled` — is on.)

Module	Panel file (<code>frontend/src/components/superadmin/</code>)	Dashboard tab id
Foundation	<code>CompetencyFrameworkIntelligencePanel.tsx</code>	<code>cmp-framework-intel</code>
1.1 Type	(no dedicated panel — surfaced inside the Framework Intelligence / Master panels)	—
1.2 Master	<code>CompetencyMasterPanel.tsx</code>	<code>cmp-master</code>
1.4 Micro	<code>CompetencyMicroFrameworkPanel.tsx</code>	<code>cmp-micro-framework</code>
1.5 Role Profile	<code>RoleCompetencyProfilePanel.tsx</code>	<code>cmp-role-profile</code>
1.6 Assessment Foundation	<code>AssessmentFoundationMappingPanel.tsx</code>	<code>cmp-assessment-mapping</code>
1.7 Search	<code>CompetencySearchPanel.tsx</code>	<code>cmp-search-discovery</code>

User-facing paths (indirect consumers of the curated genome)

There is **no per-module user page** for Phase 1.x — the modules are super-admin curation tools. The genome they curate is read by the end-user competency experience through the separate `/api/competency/*` runtime API. Each user page is an SPA screen: navigate by setting the screen value (the app routes on `currentScreen` in `frontend/src/App.tsx`), e.g. `career-builder` or `career-builder?tab=assessment`.

User page	SPA screen (set <code>currentScreen</code>)	File
Career workspace	<code>career-builder</code>	<code>frontend/src/pages/CareerBuilderPage.tsx</code>
Competency intelligence home	<code>competency-intelligence</code>	<code>frontend/src/pages/CompetencyIntelligencePage.tsx</code>
Gap analysis	<code>competency-gap-analysis</code>	<code>frontend/src/pages/competency/GapAnalysisPage.tsx</code>
Industry benchmarks	<code>competency-benchmarks</code>	<code>frontend/src/pages/competency/IndustryBenchmarksPage.tsx</code>
Career stages	<code>competency-career-stages</code>	<code>frontend/src/pages/competency/CareerStagePage.tsx</code>
Role transition	<code>competency-role-transition</code>	<code>frontend/src/pages/competency/RoleTransitionPage.tsx</code>
Hiring prediction	<code>competency-hiring-prediction</code>	<code>frontend/src/pages/competency/HiringPredictionPage.tsx</code>
Growth simulation	<code>competency-growth-simulation</code>	<code>frontend/src/pages/competency/GrowthSimulationPage.tsx</code>
Learning paths	<code>competency-learning-paths</code>	<code>frontend/src/pages/competency/LearningPathsPage.tsx</code>
Frameworks overview	<code>intelligence-frameworks</code>	<code>frontend/src/pages/IntelligenceFrameworksPage.tsx</code>
Competency dashboard (component)	rendered within the above	<code>frontend/src/components/CompetencyDashboard.tsx</code>

Honesty note: these pages consume the **runtime** competency API (`/api/competency/*`), so they reflect the genome curated by the Phase 1.x modules but are not the modules themselves. The Phase 1.x admin endpoints (`/api/competency-intelligence/*`) are called **only** by the super-admin panels above.

Data Import Options (how data gets into each module)

There are **five** ways data enters the Phase 1.x modules. Not every module supports every method — the matrix at the end states exactly which apply. **Nothing here ever fabricates data**: seeds are idempotent and additive, imports are **ON CONFLICT** upserts, and empty tables stay empty (and are reported honestly) until you import.

Option 1 — Seed scripts (primary; per module)

Run from the repo root: `cd backend && npx tsx scripts/<seed>.ts`. Seeds are **idempotent** (safe to re-run) and **additive** (never duplicate or overwrite the genome). A workflow/route restart is **not** needed after a seed (only after adding a new route). Re-run the matching admin **summary/report** afterwards to confirm

the rows landed.

Module	Seed command	What it imports / derives	Source of truth
1.1 Type	<code>npx tsx scripts/seed-competency-types.ts</code>	A type (1 of 5) for each of the 299 competencies	deterministic family/lexicon rules over the genome
1.2 Master	<code>npx tsx scripts/seed-competency-master.ts</code> (+ <code>seed-master-fixups.ts</code>)	Status + 6 eligibility flags per competency	curated defaults; fixups patch known rows
1.4 Micro	<code>npx tsx scripts/seed-micro-competency.ts</code>	Parent→child micro relationships	curated micro map
1.5 Role Profile	<code>npx tsx scripts/seed-role-competency-profile.ts</code>	Per-role competency requirements (level/weight/criticality)	curated role profiles
1.6 Assessment Foundation	<code>npx tsx scripts/seed-assessment-foundation.ts</code>	Blueprints (derived from 1.5), role→assessment, competency→question links	projected from Phase 1.5 + the question bank
(questions)	<code>npx tsx scripts/seed-competency-templates.ts</code>	Competency question templates (so 1.6 has questions to link)	curated templates
1.7 Search	<i>(none — reads the other modules)</i>	—	—

If a seed's prerequisite is empty, the seed honestly does nothing for that part (e.g. 1.6's competency→question links stay empty until questions exist in `competency_question_templates`). That is by design — it is never seeded with placeholder questions.

Option 2 — Foundation genome import (SQL migration + O*NET)

The 299-competency genome itself is **not** runtime-seeded; it is curated via SQL migration (the `onto_*` library, applied by the post-merge migration runner). To expand the role/skill library from the public-domain **O*NET** dataset (populates the `ont_*` namespace, disjoint from the curated `onto_*` starter rows):

```
cd backend && npx tsx scripts/onet-import-run.ts # downloads + imports
cd backend && npx tsx scripts/onet-import-run.ts --no-download # use cached files only
cd backend && IMPORTANCE_THRESHOLD=2.5 npx tsx scripts/onet-import-run.ts # tune cutoff
```

This importer is idempotent (`ON CONFLICT DO UPDATE/NOTHING`) and additive. In dev the `ont_*` tables are intentionally empty; run this in production to populate the full O*NET-derived library.

Option 3 — Bulk file upload (Upload Service · XLSX/CSV)

The separate **Upload Service** (FastAPI app `backend-main/`, port **8000**, its own workflow, shares the same database) is the bulk file-import path for the **question bank** that the assessment surface (1.6) links against.

- **UI:** GET `http://localhost:8000/admin/upload` (renders an upload form, `app/templates/upload.html`).
- **API:** POST `http://localhost:8000/admin/upload` with `multipart/form-data` — a file (XLSX/CSV) plus `upload_type`.
- **upload_type values:** `question_bank`, `question_options`, `task_variants`.
- **Columns are normalised** (case/space/-/_ insensitive) with built-in aliases, e.g. `QuestionCode`→`question_code`, `QuestionText`→`question_text`, `QuestionType`→`question_type`, `OptionScore`→`option_score`, `IsCorrect`→`is_correct`, `AgeBand`→`age_band`. Rows are validated per type, blanks become `NULL`, and inserts are **idempotent upserts**.
- **First-time setup:** POST `http://localhost:8000/admin/bootstrap` creates the uploader's own tables.

Honesty note: the Upload Service writes the uploader's question-bank tables. Once real questions exist in `competency_question_templates`, re-run the 1.6 seed (`seed-assessment-foundation-mapping.ts`) to derive the competency→question links. The CFI super-admin panels themselves do **not** accept file uploads — they are form-based (single-row) editors.

Option 4 — Manual entry via the super-admin panels / CRUD API

For one-off additions and corrections, use the panel forms (or call the admin endpoints directly). All validate that referenced roles/competencies/questions already exist and never create a competency out of thin air:

Module	Manual create/edit	Endpoint(s)
1.2 Master	edit status + 6 flags	<code>PATCH /api/admin/competency-intelligence/master/ :id</code>
1.4 Micro	add / toggle / reorder / delete relationships	<code>POST · PATCH /:id · DELETE /:id</code> (micro-framework)

1.5 Role Profile	add / edit / delete role requirements	POST · PATCH /:id · DELETE /:id (role-profiles)
1.6 Assessment Foundation	manage blueprints / blueprint-competencies / role-assessments / competency-questions	their respective POST/PATCH/DELETE

Option 5 — Bulk update (not import) — Search & Discovery

1.7's `POST /api/admin/competency-intelligence/search/bulk` runs across a **filtered result set** of existing competencies. It supports exactly two operations: `export` (read out the selection) and `assign_type` (set the Phase 1.1 type axis in bulk, stamped `provenance='manual_bulk'`). It does not change master status and does not import new competencies.

Import-support matrix

Module	Seed	SQL/O*NET genome	Bulk file upload	Manual CRUD	Bulk update
Foundation	—	[OK]	—	—	—
1.1 Type	[OK]	—	—	— (seed-driven)	[OK] (1.7 <code>assign_type</code>)
1.2 Master	[OK]	—	—	[OK]	—
1.4 Micro	[OK]	—	—	[OK]	—
1.5 Role Profile	[OK]	—	—	[OK]	—
1.6 Assessment Foundation	[OK]	—	[OK] (question bank, via Upload Service)	[OK]	—
1.7 Search	—	—	—	—	[OK] (acts on others)

User-manual: managing a module step by step

The generic loop is the same for every CRUD module (1.2 / 1.4 / 1.5 / 1.6). Foundation and 1.1 are read-only/seed-driven; 1.7 is search + bulk.

Generic workflow (CRUD modules):

- Open the panel** — log in as super admin → `admin-dashboard` → open the module's tab (id in the table above).
- Review current state** — the panel loads the live rows and the module's **summary report** (coverage + honest findings). Empty? It tells you which seed to run.
- Import in bulk** — run the module's seed (Option 1), or for 1.6 questions use the Upload Service (Option 3). Click **Refresh** in the panel to reload.
- Adjust manually** — use the form to add a row, or the inline controls to edit/toggle/delete (Option 4). Validation blocks references to non-existent roles/competencies.
- Verify** — re-read the summary report; confirm coverage rose and no new honest-gap findings appear (e.g. 1.6 warns if blueprint weights don't sum to 100 — fix at the source, it is never auto-normalised).
- Roll-up check** — open the Framework Intelligence panel (`cmp-framework-intel`) → the **readiness** report should now show the module's table as `consumable` rather than `empty_pending_import`.

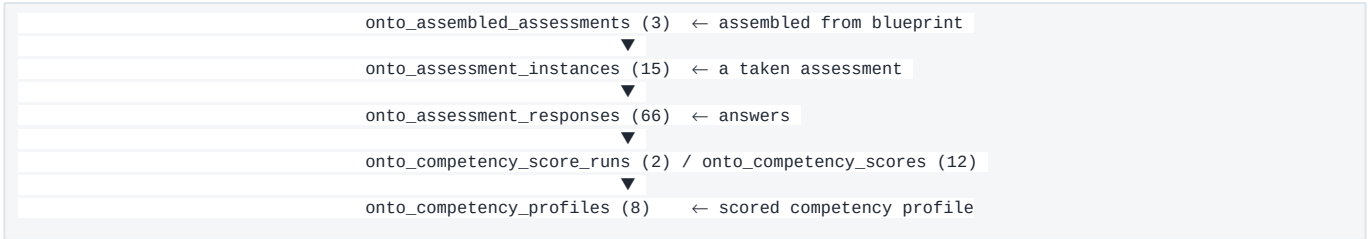
Per-module specifics:

- **Foundation (`cmp-framework-intel`)** — read-only. Use it to monitor the readiness roll-up across all sub-phases; change genome content via migration only.
- **1.1 Type** — no per-row editor; re-run `seed-competency-types.ts` after rule/genome changes, then review `needs_review` in the classification report.
- **1.2 Master (`cmp-master`)** — flip the 6 eligibility flags / status per competency to control which downstream products may consume it; `source` flips to `curated` when you edit.
- **1.4 Micro (`cmp-micro-framework`)** — build the parent→child tree; a child can be a linked existing competency or a named-only micro (honestly flagged).
- **1.5 Role Profile (`cmp-role-profile`)** — set each role's required level/weight/criticality; the matrix and weighted role-readiness recompute from these.
- **1.6 Assessment Foundation (`cmp-assessment-mapping`)** — derive blueprints from 1.5, map roles→assessments and competencies→questions; watch the weight-integrity finding.
- **1.7 Search (`cmp-search-discovery`)** — faceted search across all layers; use bulk operations to apply one change to a filtered set.

End-to-end runtime sequence (how an assessment actually flows)

The Phase 1 foundations are consumed downstream by the live assessment runtime:

```
[1.1] type -■
[1.2] master(eligible)-■-> [1.5] role profile -derive-> [1.6] blueprint + blueprint_competency_map
[1.4] micro -■
```



This proves data has flowed through the foundations end-to-end (not just structurally present) — the runtime tables are populated.

How the report is generated (post-assessment)

Once a person finishes the assessment, **there is no separate "generate report" button**. The report is the scored result — it is produced the instant scoring runs, and then read back on demand.

What happens, step by step

1. **The person submits answers** → stored in `onto_assessment_responses`.
2. **Scoring runs automatically**. Two scorers can write the result:
 - `backend/services/competency-scoring.ts` — the rich normalised scorer → writes one row to `onto_competency_score_runs`.
 - `backend/services/competency-runtime.ts (scoreAssessment)` — the runtime path → writes the **competency profile** to `onto_competency_profiles` (append-only: one row per scoring run, never edited in place).
3. **The profile is assembled** — overall score + level, a per-domain breakdown, a coverage figure, and honest "what we couldn't measure" notes.
4. **Role readiness is derived** by `backend/services/role-competency-profile.ts`: for each competency, $\text{attainment} = \min(\text{actual} \div \text{required}, 1)$; the role score = $\Sigma(\text{attainment} \times \text{weight}) \div \Sigma(\text{weight of assessed competencies}) \times 100$. If any *critical* competency falls below its required level it is flagged as a **blocking gap**, and the overall role fit is capped at "partial" no matter how high the number.
5. **The report is read back** — per person — through the competency-runtime endpoints (`backend/routes/competency-runtime.ts`):
 - `GET /api/competency-runtime/profiles/:subjectId` — the latest scored competency profile for one person.
 - `GET /api/competency-runtime/gap-analysis/:subjectId` — required vs measured gaps for that person.
 - `GET /api/competency-runtime/score-runs/:runId` — the raw score-run ledger entry.
 - (Separate, framework-level analytics — not a per-person report — live at `GET /api/admin/competency-intelligence/readiness and /classification-report` in `backend/routes/competency-intelligence.ts`.)

What the report actually contains — a real example

Pulled live from `onto_competency_profiles` (generated 2026-06-19):

Field	Value
Subject	<code>adaptive_smoke_1</code>
Role assessed	<code>role_pm</code> (Product Manager) · blueprint <code>bp_pm_v1</code>
Overall	72.5 → Level 4
Competencies in blueprint	6 (5 measurable, 1 unmeasurable)

Per-domain breakdown (the scored body of the report):

Domain	Level	Score	Questions answered
Behavioral Capabilities	5	100	1
Interpersonal & Leadership Capabilities	3	45	5

Honesty notes carried *inside* the report (Coverage vs Confidence in action):

- [WARN] "1 of 6 blueprint competencies are UNMEASURABLE — no question-bank coverage for their onto-domain." The competency **Agile Collaboration** maps to domain `dom_strategic`, which has no questions, so it is left **unscored** rather than guessed.
- [WARN] "Per-competency scores are a domain-PROXY" — the 7-code question bank crosswalks down to 5 domains until `onto_competency_question_map` is populated. The score is honestly labelled as a proxy.

How to see it yourself

- **As super-admin (one person's report):** log in, then open `$REPLIT_DEV_DOMAIN/api/competency-runtime/profiles/<subjectId>` (e.g. `adaptive_smoke_1`) — that returns the latest scored profile. Pair it with `/api/competency-runtime/gap-analysis/<subjectId>` for the required-vs-measured gaps.
- **As super-admin (whole-framework health):** `$REPLIT_DEV_DOMAIN/api/admin/competency-intelligence/readiness` — note this is framework-level analytics, not a single person's report.
- **Straight from the data:** query `onto_competency_profiles` (the `profile` column is the full per-domain JSON) — that is the same content the report is built from.

Note: a report only has content if a session was actually completed and scored. The competency runtime tables hold real rows today; the separate **CAPADEX behavioural** stakeholder report (`backend/services/pil/report-builder.ts`, `GET /api/capadex/session/:id/report`) is empty in the live DB because there are currently **0 completed CAPADEX sessions**.

Summary table

Phase	Module	Public reads	Admin CRUD	Tables (live rows)	State
1	Foundation spine	spine/competencies/levels/individuals	index/assess/economy/crosswalk	onto_competencies 299 + taxonomy	[OK] active
1.1	Type Classification	types, type-map	classification-report	onto_competency_types 5 / type_map 299	[OK] active (content gap: future/technical)
1.2	Master Enhancement	master	master-summary, PATCH master/:id	onto_competency_master_ext 0	[WARN] empty in live DB
1.4	Micro Competency	micro-framework, micro-mapping	summary + POST/PATCH/DELETE	onto_competency_hierarchy 0	[WARN] empty in live DB
1.5	Role Competency Profile	role-profiles, role-matrix, role-readiness	summary + CRUD	role_competency_profiles 14 / role_weights 35	[OK] active (breadth gap)
1.6	Assessment Foundation	blueprints, role-assessments, competency-questions	summary + full CRUD + derive	blueprints 6 / blueprint_comp 28 / role_assess 3 / question_map 25 (7 distinct comps)	[OK] active (7/299 measured)
1.7	Search & Discovery	search, facets, micro-search	bulk operation	(reads all above)	[OK] active

1.3 exists only as a one-off data-quality audit (`reports/competency_quality_report.md`), not as code; 1.8–1.15 do not exist. Empty tables (1.2, 1.4) are reported as empty, not inflated.

End of Part 1. Everything above is the framework set-up (the back office). Everything below — Parts 2 to 6 — is what happens once a real person takes the assessment, and how the platform turns that into employability insight, career guidance, employer tools, and a commercial product.

PART 2 — PHASE 2: Competency Runtime (taking & scoring the assessment)

Flag: `FF_COMPETENCY_RUNTIME=1` (internally `competencyRuntime`) · **Routes:** `backend/routes/competency-runtime.ts` (`/api/competency-runtime/*`) · **Core service:** `backend/services/competency-runtime.ts`.

In plain English

Phase 1 built the *blueprint*. **Phase 2 is the live machine that runs it.** A person sits down, is given a set of questions assembled from a blueprint, answers them, and the engine turns those answers into a **competency profile** — an overall score, a level (1–5), a per-domain breakdown, and an honest note of anything it *couldn't* measure. This is the first phase where **real human data exists** in the system.

blueprint (Phase 1.6) -> ASSEMBLE questions -> person ANSWERS -> SCORE -> competency profile (+ role readiness)
2.1 dimension mix 2.2 question map 2.3 assembly 2.4 scoring (the report)

What each audience does

■ End user (the person being assessed)

- Opens an assessment, answers a balanced, role-relevant set of questions (harder questions quietly count for more).
- At the end sees a **competency profile**: overall score → level, a breakdown by skill domain, and plain "what we couldn't measure yet" notes (it never guesses a score it doesn't have).
- Can re-take later; each run is saved as its own row (history is never overwritten).

■ Admin / super-admin

- Monitors assessment instances and a subject's profile history.
- Reviews the **scoring audit** (how raw answers became a normalised score) and confirms the engine respected the honesty rules (unmeasurable skills left unscored, not floored to 1).
- Manages the difficulty ladder and the question→competency mapping that feed assembly.

■ Developer

- **Two scorers, two ledgers** — `scoreAssessment` (in `competency-runtime.ts`) writes the **competency profile** to `onto_competency_profiles` (append-only, one row per run); the richer normalised scorer (`competency-scoring.ts`) writes a row to `onto_competency_score_runs`. Any "how many were scored" count must UNION both tables, or runtime-scored people look unscored.
- Scoring is currently a **domain proxy**: the 7-code question bank crosswalks down to the 5 ontology domains and scores the mean; it **auto-upgrades** to true per-competency scoring once `onto_competency_question_map` is populated. The domain `dom_strategic` has no questions, so it is reported **UNMEASURABLE**, never scored.
- `subject_id` is **operator-supplied**, so the runtime endpoints must sit behind the super-admin gate (a `requireAuth`-only route here would be an IDOR).
- Difficulty-weighted scoring (2.4) maps a normalised score to a 1–5 level on fixed bands; reverse-scored (polarity) questions invert at generate time.

Sub-phases (file-header markers)

#	Name	What it adds	Key file	Tables it owns	Main endpoint
2 (core)	Competency Runtime	Create an assessment instance, submit answers, get a scored profile	<code>services/competency-</code>	<code>onto_competency_profiles</code> <code>onto_assessment_instances</code> <code>onto_assessment_responses</code>	<code>POST /api/competency-runtime/assessment</code> <code>GET /profiles/:subjectId</code> <code>GET /gap-analysis/:subjectId</code>
2.1	Blueprint dimension mix	Per-role "% mix" across competency types (e.g. 40% cognitive / 30% behavioural), must sum to 100	<code>services/blueprint-builder</code>	<code>onto_blueprint_dimensions</code>	<code>GET /blueprints/:id/mix</code>
2.2	Question blueprint	Map question-bank items → competencies + a 1–5 difficulty ladder	<code>services/question-blueprint</code>	<code>onto_competency_questions</code> difficulty/blueprint tables	<code>GET /questions/difficulty-framework</code>
2.3	Assessment assembly	Pick + randomise a balanced, duplicate-free set of questions from a blueprint	<code>services/assessment-assembler</code>	<code>onto_assembled_assessments</code>	<code>POST /assembly/build</code>
2.4	Scoring engine	Difficulty-weighted score → 1–5 proficiency level + scoring audit	<code>services/competency-scoring</code>	<code>onto_competency_score_runs</code>	<code>GET /scoring-runs/:id</code>

Developer note (assembly honesty): each question is assigned to **exactly one** competency (highest weight; ties broken by id ascending), so duplicates are structurally impossible. Randomisation uses a seeded shuffle so a run is reproducible.

Honest live status

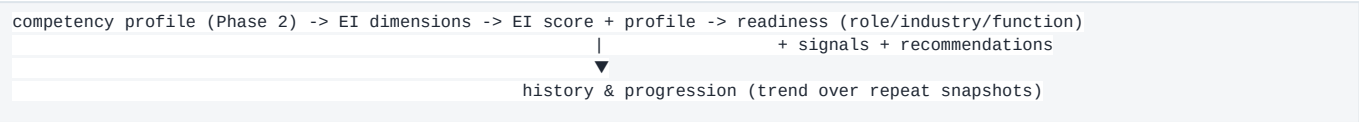
[OK] **Proven end-to-end.** The live dev database holds real rows: ≈8 competency profiles (`onto_competency_profiles`) and ≈2 score runs (`onto_competency_score_runs`) from completed assessments — this is the proof that data actually flows from Phase 1 blueprints through scoring. [WARN] **Thin coverage:** because only 7 of 299 competencies have question links yet (Phase 1.6 gap), scores are domain-proxies and one domain (`dom_strategic`) is honestly unmeasurable.

PART 3 — PHASE 3: Employability / EI Intelligence

Flag: `FF_COMPETENCY_EI=1` (internally `competencyEi`), with `FF_COMPETENCY_INTELLIGENCE=1` · **Routes:** `/api/competency-ei/*` · **Nature:** read-only, compose-only — it never re-scores; it *re-shapes* the Phase 2 profile into employability meaning.

In plain English

Phase 2 tells you *how good someone is at each skill*. **Phase 3 answers "so how employable are they, and what should they do next?"** It rolls the competency profile up into an **Employability Index (EI)** across a fixed set of dimensions, then adds readiness against roles / industries / functions, qualitative **signals** ("strong problem-solver"), and a **growth path** of recommendations — and tracks all of it over time.



What each audience does

- End user
- Sees a single **Employability Index** plus the dimensions behind it, their strengths and risks, role/industry/function fit, plain-language **signals**, and a **growth path** of concrete actions.
 - Over repeat assessments, sees an **employability journey** (improving / stable / declining) — only once there are **two or more measured snapshots**; with one, it honestly says "not enough history yet".
- Admin
- Curates the **weights** that drive the EI index, the **signal rules**, and the **recommendation library** (recommendations are library-backed — the system never invents generic advice).
 - Uses the **EI dashboard** (aggregate gauges + trends) and the **super-admin validation harness** to confirm the chain is honest (no fabricated scores, flags respected, bands coherent).
- Developer
- **One EI formula authority** lives in `employabilityEngine.ts`; classifiers must not be re-implemented inline or the gauge and the backend score drift apart.
 - Every engine **composes** prior outputs — e.g. the EI dashboard (3.10) calls each prior engine **once** then makes pure projections; the trend math (3.11) lives in **one trend-engine.ts** that both history and dashboard import.
 - History has **two sources**: assessment runs and **EI snapshots** (`ei_profile_snapshots`, append-only, written only on the POST/snapshot path — a GET must never write). A missing value is **NULL**, **never 0** (a fake 0 would invent a data point and flip a trend).
 - Risk-style dimensions invert direction (higher is worse); readiness ≥ 85 caps "role potential" to Low (a positive signal, done in code, not a bug).

Sub-phases (3.1 does not exist — the series starts at 3.2)

#	Name	What it adds	Key file
3.2	Competency → EI dimension mapping	Translate competencies into employability dimensions + weights	<code>services/competency-ei-dimensions.ts</code>
3.3	Employability scoring engine	Per-dimension + aggregate EI score (3 pure engines + audit ledger)	<code>services/employability-scoring-engine.ts</code>
3.4	EI Profile	Unified, versioned candidate employability profile (strengths/risks/index)	<code>services/ei-profile-engine.ts</code>
3.5	Role Readiness V2	Adds role risk + role potential on top of readiness	<code>services/role-readiness-v2.ts</code>
3.6	Industry Readiness	Fit/gap vs industry benchmarks (demand derived by aggregating role profiles)	<code>services/industry-readiness-engine.ts</code>
3.7	Function Readiness	Same idea, one hop shorter — vs business-function benchmarks	<code>services/function-readiness-engine.ts</code>
3.8	Employability Signal	Rule-based qualitative signals from the profile	<code>services/employability-signal-engine.ts</code>

3.9	Recommendation Engine	Library-backed growth actions (emitted / not-applicable / withheld)	services/ei-recommendation-engine.ts
3.10	EI Dashboard	Composes every prior EI engine into candidate + admin views	services/ei-dashboard-engine.ts
3.11	EI History / Progression	Longitudinal trend (improving/stable/declining) over snapshots	services/progression-engine.ts (math in trend-engine.ts)
3.12	Super-Admin Validation	10-area read-only honesty harness across the whole EI chain	services/super-admin-validation-engine.ts

Main endpoints: [GET /api/competency-ei/intelligence/:subject](#) (the full envelope) · [POST /intelligence/:subject/snapshot](#) (write a snapshot) · [GET /intelligence/:subject/history](#) · [GET /admin/overview](#) · [GET /super-validation/:subject](#) (the 3.12 super-admin honesty harness — distinct from [/validation/:subject](#), which is the narrower per-chain validation).

Honesty rules baked in: Coverage (data exists) and Confidence (enough measured signal) are reported separately; [dom_strategic](#) is excluded; recommendations are **withheld** rather than guessed when evidence is missing; trends need ≥ 2 *measured* points.

Honest live status

[ENGINE] **Built and flag-on, but only as rich as the runtime beneath it.** These engines re-shape the ≈ 8 Phase-2 profiles, so EI output exists but is sparse, and progression is mostly "insufficient history" because repeat snapshots are rare in the dev DB. The validation harness (3.12) is the tool that reports exactly which areas are measured vs honestly absent.

PART 4 — PHASE 4: Career Intelligence

Flags (the [FF_CAREER_*](#) family, all on in the workflow): [FF_CAREER_MATCH](#), [FF_CAREER_READINESS](#), [FF_CAREER_GAP](#), [FF_CAREER_ROADMAP](#), [FF_CAREER_DEVELOPMENT](#), [FF_CAREER_RECOMMENDATION](#), [FF_CAREER_SIGNAL](#), [FF_CAREER_PROGRESSION](#), [FF_CAREER_SIMULATION](#), [FF_CAREER_PASSPORT_FOUNDATION](#), [FF_CAREER_VALIDATION](#). · **Surface:** the Career Builder page + [/api/career/*](#) (enrichment) and dedicated aggregator routes. · **Nature:** **compose-only**, **flag-OFF** byte-identical, **GET-never-writes**.

In plain English

Phase 3 says *how employable* a person is. **Phase 4 turns that into a career plan.** It matches the person to roles, shows exactly which skills are short (the gap), builds a step-by-step **roadmap**, lets them **simulate** "what if I grew this skill", tracks **progress**, and rolls everything into a portable **career passport**. It is purely additive: when the flags are off, the Career Builder behaves exactly as it did before — these engines only *attach* extra intelligence to data that already exists.

EI profile (Phase 3) -> MATCH to roles -> GAP vs target role -> ROADMAP -> track PROGRESS / SIMULATE growth -> PASSPORT

What each audience does

■ End user (job-seeker)

- Sees **role matches** (ranked), the **gap** to a target role, a **development roadmap**, "what-if" **growth simulations**, **progress** over time, and developmental **signals**.
- Gets a **career passport** — a portable summary they can share (contact details are never published).

■ Admin

- Curates role catalogues and recommendation content; monitors the validation harness output.
- Note: most Phase-4 insight is **derived**, not hand-entered — admins mostly maintain the underlying role/competency data from Phase 1.

■ Developer

- Every 4.x engine is **compose-only** — it re-shapes prior data and **never recomputes** scores; flag-OFF must be **byte-identical** and **GETs must not write**.
- **Trap:** [buildCareerReadiness](#) → [computeRoleReadinessV2](#) runs [ensureCompetencyRuntimeSchema](#) *unguarded* (it does DDL), so any GET composer must gate behind a [competencyRuntimeReady\(\)](#) probe first.
- The enrichment pattern attaches a flag-gated [career_intelligence](#) key onto the **existing** career response; [resolveEffectiveUserId](#) is the IDOR guard and must be applied at the **route** level too, not just on the enrichment.
- [career_seeker_profiles](#) PK is [user_id](#) (there is no [id](#) column); data is JSONB.
- **Honesty traps:** [cg_roles](#) has no per-role requirements, so only the **anchor** role match is requirement-backed — all other matches are capped at **Provisional** (Match% and confidence are *separate* axes). [Number\(null\) === 0](#) would fabricate a measured 0, so signals guard null/"undefined" before any [Number\(\)](#).

Sub-phases (4.1, 4.4, 4.5, 4.8, 4.9 are not distinct file-marked engines; the confirmed set:)

#	Name	What it does	Flag
4.2	Career Match	Rank roles by fit; anchor match is requirement-backed, rest Provisional	FF_CAREER_MATCH
4.3	Career Readiness aggregator	Compose readiness across competency/EI/future-readiness blocks	FF_CAREER_READINESS
(gap)	Career Gap	Required-vs-measured gap to a target role	FF_CAREER_GAP
(roadmap)	Career Roadmap	Ordered development milestones	FF_CAREER_ROADMAP
4.6	Career Development	Develop skills via the 5 real competency types (no fabricated "Leadership" type)	FF_CAREER_DEVELOPMENT
4.7	Career Recommendation	Personalised vs catalog recs (catalog-only → always Provisional)	FF_CAREER_RECOMMENDATION
(simulation)	Career Simulation	"What if I improve X" projections	FF_CAREER_SIMULATION
4.10	Career Signal	7 developmental signals (null-safe, never fabricated)	FF_CAREER_SIGNAL
(progression)	Career Progression	Track movement over time	FF_CAREER_PROGRESSION
(passport)	Career Passport (foundation)	Portable career summary (contact never published)	FF_CAREER_PASSPORT_FOUNDATION
4.12	Career Validation harness	13-area read-only honesty harness	FF_CAREER_VALIDATION

The standalone numbers shown in parentheses are flag-named modules without a confirmed 4.x file-header marker — listed by flag for accuracy rather than asserting a number that may not exist in code.

Honest live status

[ENGINE] **Compose-only and flag-on.** Phase 4 surfaces inside the existing Career Builder for any user who has runtime data underneath them. With the dev DB's sparse runtime, matches/gaps/roadmaps populate where there is data and degrade honestly (Provisional / "unmeasured") where there isn't — they never fabricate a match or a gap.

PART 5 — PHASE 5: Employer / Enterprise

Flags: FF_EIOS_WORLD_CLASS_VERIFIED_V2 (Employer Intelligence OS), FF_ENTERPRISE_ANALYTICS, FF_AI_GOVERNANCE, FF_GOVERNANCE_RBAC_V2. · **Spine:** employer_* (≈7 tables) and tig_* (Talent Intelligence Graph) tables; LBI engine. · **Auth:** employer endpoints are **session-only** (requireAuth), so demo data is seeded server-side and marked @example.com.

Numbering honesty: unlike Phases 1–4, the employer band is organised by **module**, not by clean file-header 5.x markers. The build logs reference numbers like 5.3 (job posting), 5.4 (talent discovery), 5.5 (competency matching), 5.11 (hiring intelligence), 5.12 (workforce intelligence) and **5.14 (notifications & workflows)** — these are documented below by **name** so nothing is over-claimed.

In plain English

Phases 1–4 are about the **individual**. **Phase 5 is the employer's side of the marketplace.** Employers post jobs, search a talent pool, see how candidates match their roles, run predictive **hiring intelligence**, look 36 months ahead at **workforce / skill-demand** trends, and view it all through a 28-pillar "cockpit" dashboard with exportable PDF reports.

What each audience does

- **End user (here, the employer/recruiter)**
- Posts and versions **jobs**; searches and pools **talent**; opens a **candidate drawer** that shows **Coverage** (which skill domains are covered) and **Confidence** (how calibrated the prediction is) as *separate* figures.
 - Runs **hiring intelligence** (interview substrate, operator-recorded outcomes) and **workforce forecasts**; generates PDF reports from the Report Factory.
 - Sees the **Talent Intelligence Graph** — an org/skill network view.
- **Admin / super-admin**
- Validates hiring-intelligence row counts, manages enterprise feature-flag gating, and runs governance/RBAC (roles, approvals, audit trail, security posture) under `FF_GOVERNANCE_RBAC_V2`.
 - Manages AI-governance and enterprise-analytics surfaces.
- **Developer**
- Every engine is **compose-only** (reads the substrate, no destructive recompute).
 - **Traps**: a column named `values` must be `values_list` (reserved keyword); the M5 route family needs `app.use('/api/m5', requireAuth)` because the per-route gate was missing. The Talent Intelligence Graph is keyed by `user_email`; its calibration is a **write-once** snapshot (Brier/ECE kept raw; a borrowed prior never upgrades a TRUST level; it only becomes "calibrated" at ≥30 real outcomes).
 - The Employer Intelligence OS spans `eios-core.ts` (pillars 3, 6–17) + `eios-intelligence.ts` (pillars 18–28 + certification); peer benchmarks suppress below `k_min = 30`.

Modules in this band

Module	What it does	Key file	Tables
Employer Portal	Jobs, candidates, the recruiter cockpit	<code>routes/employer-*.ts</code>	<code>employer_*</code>
Talent Intelligence Graph (TIG)	Skill/candidate network + calibrated predictions	<code>routes/employer-tig.ts</code>	<code>tig_*</code>
Employer Intelligence OS (EIOS)	28-pillar intelligence + certification	<code>services/eios-core.ts</code> , <code>eios-intelligence.ts</code>	<code>eios_*</code>
LBI (Leadership/Behavioural Intelligence)	Consolidated behavioural intelligence chain	<code>routes/lbi-intelligence.ts</code>	<code>lbi_*</code>
Notifications & Workflows (≈5.14)	Workflow events, overdue-outcome nudges (no candidate PII in comms)	notifications engine	<code>employer_notifications</code> / workflow events
Governance / RBAC V2	Roles, approvals, audit, security posture	governance engines	RBAC tables

Honest live status

[ENGINE] **Structurally built, flag-on, lightly populated.** The employer engines run, but in the dev DB the meaningful data is **demo seed** (marked `@example.com`); predictive calibration stays "borrowed prior" until ≥30 real hiring outcomes exist. Coverage and Confidence are surfaced separately so an employer is never shown a confident-looking number that isn't earned.

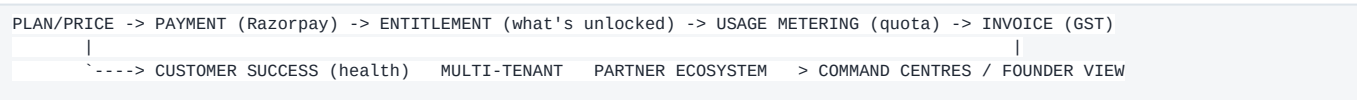
PART 6 — PHASE 6: Commercial, Platform & Super-Admin (6.1 → 6.15)

Flags: `FF_COMMERCIAL_ACTIVATION`, `FF_COMMERCIAL_ENTITLEMENT_ENFORCEMENT`, `FF_REPORT_FACTORY`, plus module flags (`commercialCustomerSuccess`, `invoiceGstEngine`, ...). · **Spine:** `capadex_payments` (one-time), `comm_subscription*` (recurring), subscription packages, `inv_invoices` (GST), `tenants`/tenant tables, partner tables. · **Nature:** the Phase-6 read engines are **read-only** and probe tables with `to_regclass` so they **degrade gracefully** instead of crashing when a table is absent.

Numbering honesty: the confirmed, code-backed `6.x` file-header modules are **6.5, 6.8, 6.10, 6.11, 6.12, 6.13 (Automation Engine), 6.14 (SuperAdmin Command Center), 6.15 (Founder Control Center)** — each is stamped at the top of its route file. **6.1–6.4, 6.6, 6.7, 6.9** are **not** distinct file-header modules — they're bundled into the Commercial Spine (catalog/pricing/entitlement) and the Report Factory. Below, numbered items are real file-header phases; un-numbered items are named.

In plain English

Phases 1–5 build the product. **Phase 6 turns it into a business.** It handles **plans and pricing**, takes **payments** (Razorpay), enforces **entitlements** (what a plan unlocks), **meters usage** against quotas, issues **GST invoices**, tracks **customer success / health**, supports **multiple tenants** (institutions, employers, partners, franchises) and a **partner/referral** programme — and rolls everything into **command centres** for the super-admin and a **founder control centre** for the top-line numbers.



What each audience does

■ End user (the paying customer)

- Sees **subscription tiers / prices** (confirmed ■299–■1499 packages), pays securely, sees their **usage vs quota**, and receives **GST invoices**.
- A partner sees their **referrals** and **commission payouts**.

■ Admin / super-admin

- Runs the business from the **command centres**: the **SuperAdmin Command Center** (system-wide posture across ~12 domains) and the **Founder Control Center** (revenue / growth / adoption / retention north-star view).
- Manages plans, entitlements, tenants, partners; reviews customer-success health and platform-intelligence metrics; configures the seller/GST profile.

■ Developer

- **Money fails closed**: payment verification requires a local↔gateway linkage (anti-IDOR); the webhook fails closed when configured; idempotency keys prevent double-grants. **Never sell into a stub** — entitlement checks return 402/503 (fail-closed), and the paid ledger is **capadex_payments** paid rows only.
- **Metering** uses three counting kinds (period-count / latest-level / credit-balance) and a **pg_advisory_xact_lock** inside a transaction so concurrent writers can't overrun a quota; reads **to_regclass**-probe rather than **ensure-schema**.
- **Credit-issue idempotency**: a per-customer row lock *serialises* but does **not** dedup — a retried refund-to-credit doubles store value unless keyed (partial unique index on **(customer_id, idempotency_key)**).
- Phase-6 GET engines are **read-only and compose** existing commercial engines into metric groups; **every unmeasurable rate is null + an explicit note**, never a fabricated 0. A **to_regclass** probe that swallows its own error must route the failure into a **degraded** flag (otherwise "unreadable" looks like "empty").
- **Flag-gate before ensure-schema** so flag-OFF does zero DDL (byte-identical).

Modules in this band

#	Module	What it does	Key file	Flag
—	Commercial Spine	Catalog, pricing, payments (Razorpay), one-time vs recurring	routes/capadex-payments.ts, commercial services	FF_COMMERCIAL_ACTIVATION
—	Entitlement Enforcement	requireEntitlement guard; what a plan unlocks; fail-closed	entitlement engine	FF_COMMERCIAL_ENTITLEMENT_ENFORCEMENT
—	Invoice & GST Engine	GST-correct invoices, refund/credit docs, seller config, txn-safe numbering	invoice engine	invoiceGstEngine
6.5	Usage Metering	Records metered actions, enforces quotas (fail-closed)	services/commercial/metering-commercial.ts	
6.8	Customer Success Intelligence	Engagement + retention → a health index	services/commercial/customer-commercialCustomerSuccess	
6.10	Platform Intelligence Console	Executive composite: health/growth/conversion/revenue (7 metric groups)	services/platform/platform-(admin)-intelligence-engine.ts	
6.11	Multi-Tenant Architecture	Institutions / employers / partners / franchises + hierarchy	services/tenant/tenant-managementConsole	(default OFF)
6.12	Partner Ecosystem	Referrals, agreements, commission payouts	services/tenant/partner-ecosystem	partnerEcosystem (default OFF)
6.13	Automation Engine	Read-only automation console (rules/triggers posture)	routes/automation-engine.ts	automationEngine (default OFF)
6.14	SuperAdmin Command Center	System-wide posture across ~12 domains	routes/superadmin-command-center	commandCenter (default OFF)

6.15	Founder Control Center	Revenue / growth / adoption / retention north-star view	routes/founder-control-centfounderControlCenter (default OFF)
—	Report Factory	PDF report generation (k=30 benchmark suppression)	services/pdf-renderer.ts, rf_templates, FF_REPORT_FACTORY

Partner payout honesty (6.12): a payout is `commission_amount` on **converted** referrals only; if only a percentage is stored, the deal value is resolved from real payment events (paise → rupees) — and if it can't be resolved, that's an honest coverage gap, never a fabricated number.

Honest live status

[AMBER] **Built and flag-on, but commercially un-activated in the dev DB.** The machinery is real and fail-closed, but there are **0 real sales** (subscription packages have no purchases) and **0 completed CAPADDEX sessions**, so **Activation** metrics read ~0 while **Structural** completeness is high. This is the dual-axis honesty rule in action: *coverage exists, activation hasn't happened yet*. The generated business reports under `reports/` (e.g. commercial / customer-success / platform-intelligence) reflect exactly this — real structure, honest zeros where nothing has been sold or completed yet.

Grand summary — the whole journey on one page

Phase	Band	Flag(s)	Proves	Honest live status
1 (+1.1–1.7)	Competency framework	FF_COMPETENCY_FRAMEWORK_INTL	The skillset → role blueprints	[OK] active (1.2/1.4 empty, 1.6 thin)
2 (2.1–2.4)	Competency runtime	FF_COMPETENCY_RUNTIME	A person is assessed & scored	[OK] real data (=8 profiles)
3 (3.2–3.12)	Employability / EI	FF_COMPETENCY_EI (+FF_COMPETENCY_INTELLIGENCE)	Profile → employability index signals + recs	[ENGINE] compose-only; as rich as the runtime
4 (4.2–4.12)	Career intelligence	FF_CAREER_* family	Match / gap / roadmap / progress	[ENGINE] compose-only, flag-OFF byte-identical
5	Employer / enterprise	FF_EIOS_WORLD_CLASS_VERIFICATION, FF_ENTERPRISE_ANALYTICS, FF_GOVERNANCE_RBAC_V2, FF_AI_GOVERNANCE	Jobs, talent, hiring & workforce intel	[ENGINE] structural; demo-seed data
6 (6.5/6.8/6.10/6.11/6.12/6.13/6.14/6.15)	Commercial / platform / per-admin	FF_COMMERCIAL_ACTIVATION, FF_COMMERCIAL_ENTITLEMENT, FF_REPORT_FACTORY on; tenantManagementConsole/partnerEcosystem/automationEngine by default; FounderControlCenter	Plans, payments, quotas, invoices, tenants, command centres	[AMBER] commercial spine flag-on but activation ~0 (no sales); 6.11–6.15 consoles flag-OFF by default

The one honesty rule that runs through all six phases: **Coverage** (does the data exist) and **Confidence / Activation** (is it trustworthy and has it actually been used) are always reported as **two separate things** — and where there is nothing yet, the system says so plainly instead of inventing a number.